


**Pannon Egyetem**
Műszaki Informatikai Kar
Rendszer- és Számítástudományi Tanszék
Programtervező informatikus BSc

# SZAKDOLGOZAT

## Megerősítéses tanulási módszerek vizsgálata az amőba variánsain

**Bánovics-Angyal Gábor**

Témavezető: Dr. Ősz Olivér

Veszprém
2026

# PANNON EGYETEM

## MŰSZAKI INFORMATIKAI KAR

### Programtervező informatikus BSc szak

Veszprém, 2022. március 23.

## SZAKDOLGOZAT TÉMAKIÍRÁS

**Hallgató neve**
Programtervező informatikus BSc szakos hallgató részére

**Szakdolgozat címe**

Témavezető: Témavezető neve, beosztása

**A feladat leírása:**

…

**Feladatkiírás:**

- Dolgozza fel a témával kapcsolatos eddigi hazai és külföldi irodalmat!
- Határozza meg a szoftverrel szemben támasztott követelményeket!
- Tervezze meg a játékhoz szükséges adatstruktúrát, algoritmusokat!
- …

Dr. Szakfelelős Oktató
egyetemi docens
szakfelelős

Témavezető Oktató
egyetemi docens
témavezető

# Hallgatói nyilatkozat

Alulírott Bánovics-Angyal Gábor hallgató kijelentem, és a dolgozat feltöltésével egyidejűleg nyilatkozom, hogy a(z) Megerősítéses tanulási módszerek vizsgálata az amőba variánsain című szakdolgozat (a továbbiakban: dolgozat) a Pannon Egyetem Rendszer- és Számítástudományi Tanszék szervezeti egységében készült a(z) programtervező informatikus oklevél megszerzése érdekében.

Kijelentem, hogy a dolgozatban csak a megadott és hivatkozott forrásokat használtam fel, és ezekre a vonatkozó idézési szabályok szerint hivatkoztam.

Nyilatkozom, hogy a dolgozat érdemi része saját szellemi alkotásom eredménye, és azt más intézményben, szakon, vagy felsőfokú képesítés megszerzésére nem nyújtottam be. Tudomásul veszem, hogy a plágium vagy szerzői jogsértés esetén a dolgozatom elutasításra kerülhet, és ellenem fegyelmi eljárás indulhat. Tudomásul veszem továbbá, hogy szerzői jogsértés esetén az Egyetem jogosult a dolgozat elérhetőségét korlátozni, valamint eltávolítani a dokumentumot a dolgozatok tárolására szolgáló, a témát vezető szervezeti egység által meghatározott elektronikus zárt rendszerből.

Tudomásul veszem továbbá, hogy a Pannon Egyetem a dolgozat eredményeit saját céljaira eltérő írásbeli megállapodás hiányában a Pannon Egyetem Szellemi Tulajdon Kezelési Szabályzatában foglaltaknak megfelelően szabadon felhasználhatja.

Nyilatkozom, hogy a dolgozat elkészítése során mesterséges intelligencia eszközöket használtam.

Nyilatkozom, hogy a dolgozat elkészítése során az alábbi táblázatban feltüntetett mesterséges intelligencia eszközöket kizárólag a kutatási, illetve fejlesztési feladat támogatására használtam fel, az érdemi munka, elemzés és következtetések teljes mértékben saját szellemi alkotásomat képezik.

A dolgozatban felhasznált mesterséges intelligencia használatot részletező táblázat:

| **Alkalmazott technológia** | **Alkalmazás módja** | **Előállított tartalom** | **MI-használat aránya** |
|---|---|---|---|
| Gemini 3.1 | LaTeX formázás és ábrák készítése | 1. fejezet | 20% |

Veszprém, 2026. november 2.

Bánovics-Angyal Gábor

# Témavezetői nyilatkozat

Alulírott Dr. Ősz Olivér témavezető kijelentem, hogy a dolgozatot Bánovics-Angyal Gábor a(z) Pannon Egyetem Rendszer- és Számítástudományi Tanszék szervezeti egységében készítette a(z) programtervező informatikus végzettség megszerzése érdekében.

Kijelentem, hogy a dolgozat védésre bocsátását engedélyezem.

Veszprém, 2026. november 2.

Dr. Ősz Olivér

# Köszönetnyilvánítás

Köszönetet mondok témavezetőmnek a szakmai iránymutatásért, a konzultációk során kapott javaslatokért és a dolgozat elkészítéséhez nyújtott támogatásért. Köszönöm továbbá mindazok segítségét, akik a fejlesztés, a tesztelés és a dokumentáció készítése során elméleti, gyakorlati vagy erkölcsi támogatást nyújtottak.

# Tartalmi összefoglaló

A diplomadolgozatom elkészítésével célom, hogy absztrakt kétszemélyes logikai játékokat vizsgáljak megerősítéses tanulási algoritmusokkal, összehasonlítva hatékonyságukat.

**Kulcsszavak:** python, numpy, gépi tanulás, megerősítéses tanulás, Q-tanulás, Monte-Carlo-fakeresés

# Abstract

**Keywords:** python, numpy, machine learning, reinforcement learning, Q-Learning, Monte Carlo tree search

# Tartalomjegyzék


**1. Bevezetés** **10**

1.1. Motiváció és a téma háttere . . . . . 10

1.2. A kutatás célja és vizsgált módszerek . . . . . 10

1.3. Szoftverarchitektúra és modularitás . . . . . 11

1.4. A dolgozat felépítése . . . . . 12

**2. Szakirodalmi áttekintés** **13**

**3. Tervezés** **14**

3.1. Absztrakt játékkörnyezet . . . . . 14

3.2. Megvalósított struktúra és függvénycsoportok . . . . . 15

3.2.1. Adattagok és inicializálás . . . . . 15

3.2.2. Lépések és játékmenet-kezelés . . . . . 15

3.2.3. Állapotkódolás . . . . . 16

3.2.4. Döntés-előkészítés és transzformációk . . . . . 16

3.3. Absztrakt játékosmodell . . . . . 17

3.3.1. A játékosmodell felépítése és feladatai . . . . . 17

3.4. Függvények specifikációja . . . . . 17

**4. Implementált játékkörnyezetek** **18**

4.1. Végtelenített amőba . . . . . 18

4.2. Megoldandó problémák . . . . . 18

4.2.1. Transzformálás . . . . . 20

4.2.2. Szimmetriák kezelése . . . . . 20

4.2.3. Lehetséges lépések terének csökkentése . . . . . 21

4.3. Állás elmentése . . . . . 21

**5. Implementált ágensek és játékosmodellek** **22**

5.1. Minimax algoritmus . . . . . 22

5.2. Monte-Carlo-fakeresésen (MCTS) alapuló ágens . . . . . 22

5.2.1. Megvalósított algoritmus . . . . . 23

**A. A mellékelt ZIP állomány könyvtárszerkezete** **24**

# 1. fejezet

# Bevezetés

## 1.1. Motiváció és a téma háttere

A mesterséges intelligencia (MI) és a gépi tanulás területe az elmúlt évtizedben drámai fejlődésen ment keresztül. Ennek a fejlődésnek az egyik leglátványosabb és legmeghatározóbb területe a megerősítéses tanulás (Reinforcement Learning, RL), amelyben az autonóm ágensek környezetükkel interakcióba lépve, jutalmak és büntetések útján tanulják meg az optimális döntéshozatali stratégiákat.

A megerősítéses tanulási algoritmusok hatékonyságának és korlátainak vizsgálatára a kétszemélyes, absztrakt logikai játékok kiváló tesztkörnyezetként szolgálnak. Ezen játékok előnye, hogy bár szigorúan meghatározott szabályrendszerrel és véges állapottérrel rendelkeznek, de a lehetséges lépéskombinációk száma miatt a tökéletes stratégia kiszámítása gyakran kivitelezhetetlen. A sakktól kezdve a Go játékig számos áttörés bizonyította, hogy az RL-alapú megközelítések képesek az emberi teljesítményt és az eddigi legjobbnak ítélt algoritmusokat is felülmúlni.

## 1.2. A kutatás célja és vizsgált módszerek

Jelen szakdolgozat célja a megerősítéses tanulási módszerek teljesítményének, konvergenciájának és hatékonyságának összehasonlító vizsgálata absztrakt kétszemélyes logikai játékokon. A kutatás fókuszában az amőba és annak egy speciális végtelenített változata áll. Az amőba ideális modell: szabályai egyszerűen leírhatók, azonban a döntési fa mélysége kellően nagy ahhoz, hogy ne lehessen egzaktul bejárni.

A dolgozat során két alapvetően eltérő megközelítést képviselő algoritmus összehasonlítására kerül sor:

- **Monte-Carlo-fakeresés** (Monte Carlo Tree Search, MCTS): Egy olyan keresési algoritmus, amely a döntési tér szimulációalapú mintavételezésével építi fel a keresési fát, és az addigi tapasztalatok alapján mélyíti a kedvező útvonalakat és egyben fedez fel újakat.
- **Q-tanulás** (Q-Learning): Egy klasszikus, értékalapú (*value-based*) modellmentes megerősítéses tanulási módszer, amely az állapot-akció párok várható jutalmát igyekszik megtanulni.

A vizsgálat során elemezzük, hogy különböző játékokban hogyan teljesítenek az egyes algoritmusok, hogyan változik a tanulási sebességük, erőforrásigényük.

## 1.3. Szoftverarchitektúra és modularitás

A kísérletek elvégzéséhez elengedhetetlen egy olyan rugalmas és bővíthető szoftverkeretrendszer tervezése és megvalósítása, amely biztosítja a különböző komponensek függetlenségét és összehasonlíthatóságát.

A megvalósítás során kiemelt szerepet kap az objektumorientált modularitás:

- **Absztrakt játékkörnyezet**: A játékok logikáját és szabályrendszerét absztrakt osztályok definiálják. Ez a struktúra kötelezően előírja a mindenkori állapotkezeléshez, a szabályos lépések lekérdezéséhez és a győzelmi feltételek ellenőrzéséhez szükséges interfészeket. Ennek köszönhetően a keretrendszerhez tetszőleges új logikai játék vagy szabályvariáció könnyen hozzáadható.
- **Absztrakt játékosmodell**: A döntéshozó egységek (ágensek) szintén egy egységes interfészt valósítanak meg. Ez lehetővé teszi, hogy a Q-tanulást végző ágens, az MCTS-alapú ágens, minimax algoritmus vagy akár egy véletlenszerűen lépő vagy egyéb szabályalapú heurisztikus algoritmus és az emberi játékos cserélhető módon, azonos feltételek mellett mérkőzhessen meg egymással.

## 1.4. A dolgozat felépítése

A szakdolgozat a következő struktúrát követi:

1. A szakirodalmi áttekintésben bemutatásra kerülnek a megerősítéses tanulás elméleti alapjai, a Q-tanulás és az MCTS működési elve.

2. A tervezést és megvalósítást részletező fejezetek az objektumorientált szoftverarchitektúrát, az absztrakt osztályok (interfészek) felépítését, valamint az implementált játékokat és ágenseket fejtik ki.

3. *A mérések és eredmények* fejezetben bemutatásra kerülnek a futtatott kísérletek, a különböző algoritmusok egymás elleni teljesítménye.

4. *A következtetések és jövőbeli munkák* fejezet foglalja össze az eredményeket és mutat rá a lehetséges továbbfejlesztési irányokra.

# 2. fejezet

# Szakirodalmi áttekintés

Megerősítéses tanulás, MCTS, Q-Learning elméleti alapjainak kifejtése.

# 3. fejezet

# Tervezés

## 3.1. Absztrakt játékkörnyezet

A keretrendszer egyik legfontosabb tervezési elve a bővíthetőség és modularitás. Annak érdekében, hogy a megvalósított megerősítéses tanulási algoritmusok (Q-tanulás, Monte-Carlo-fakeresés) függetlenek legyenek a konkrét logikai játékok szabályrendszerétől, a játékkörnyezetet egy absztrakt osztály segítségével definiáljuk. Majd a konkrét játékok ezt az osztályt fogják implementálni a salyát szabályaik alapján.

Az `abstract_game` osztály szolgál az összes kétszemélyes logikai játék közös interfészeként, amely kötelezően előírja a környezettel való interakcióhoz, a játékállapot kezeléséhez és az ágensek döntéshozatalához szükséges függvényeket.
Legfőbb feladatai, céljai:

- **Egységes interfész a játékosok számára**: A döntéshozó ágensek vagy humán játékosok kizárólag ezen az interfészen keresztül kommunikálnak a játékkal, így a játék típusa vagy szabályrendszere a döntési logika módosítása nélkül cserélhető.
- **Állapot-immutabilitás és reprodukálhatóság**: A lépések megtétele új játékállapot-példányt hoz létre, ami megkönnyíti a keresési fák (MCTS) bejárását és a tanulási állapotok visszagörgetését mellékhatások nélkül.
- **Állapotkódolás**: A játékállapotok *string* formátumra kódolhatóak és dekódolhatóak, ami elengedhetetlen a Q-táblás, illetve MCTS állapotregisztrációhoz, valamint a játékok elmentéséhez és betöltéséhez.
- **Állapottér-redukció és szimmetriakezelés**: Az osztály beépített interfészt biztosít a táblatranszformációkhoz, amelyek csökkentik a tanulási tér méretét a szimmetriák kihasználásával vagy kényszerített lépések előzetes meghatározásával.

## 3.2. Megvalósított struktúra és függvénycsoportok

Az `abstract_game` osztály az `abstract_game_display` megjelenítési interfészből és a Python `ABC` (Abstract Base Class) osztályából származik, biztosítva a megjelenítési funkciók és a logikai megvalósítás együttes meglétét és egyben elkülönítését is.

### 3.2.1. Adattagok és inicializálás

- `size` (`int`): A játékmező/tábla dimenzióját vagy teljes méretét határozza meg *1D* reprezentációnak megfelelően.
- `side` (`Side`): Beépített `IntEnum` segítségével a soron következő játékos oldalát reprezentálja játéktól függetlenül, `FIRST` és `SECOND` használata jelek vagy színek megnevezése helyett.
- `board` (`NDArray[np.int32]`): *NumPy* tömb alapú állapotreprezentáció a hatékonyabb számítás érdekében, amely a tábla aktuális állását tárolja.

### 3.2.2. Lépések és játékmenet-kezelés

A játékmenet előregördítéséért, a szabályok ellenőrzéséért és a végállapotok detektálásáért felelős függvények:

- `make_move(position)`: Végrehajtja a megadott pozíciójú lépést a táblán, és visszatér az új játékállapot-példánnyal (`Self`). Az egyes lépések megadása játékonként változik (pl. van ahol fontos, hogy mivel lépünk vagy honnanhova), ezért a lépések megadásának általános formai követelménye, hogy egy `tuple` legyen valamennyi `int` elemmel, melynek a számát az implementáló osztály rögzíti.
- `if_valid_move(move)`: Ellenőrzi, hogy az adott lépés szabályos-e.
- `get_winner()`: Meghatározza a játék győztesét a `Side` segítségével. Amennyiben a játéknak még nincs vége (vagy döntetlen az állás), -1 értékkel jelzi azt.
- `get_force_win()`: Detektálja az azonnali nyerő (kényszerített) lépéseket. Ha létezik kényszerített győzelmet jelentő lépés, visszaadja annak pozícióját, ellenkező esetben a visszatérési érték első eleme -1.
- `get_possible_moves()`: Meghatározza az aktuális állapotban megtehető összes szabályos lépést.
- `get_possible_last_positions()`: Az aktuális állapotot megelőző összes lehetséges állapotot hivatott meghatározni.

### 3.2.3. Állapotkódolás

A megerősítéses tanulási algoritmusok állapotazonosításához, valamint a játékállások elmentéséhez szükséges függvények:

- `board2str()`: Az aktuális táblaállást egyedi `string` reprezentációvá alakítja.
- `str2board(str_board)`: Osztálymetódus, amely egy korábban elmentett kódolt *string*ből rekonstruálja és példányosítja az adott `abstract_game` objektumot.

### 3.2.4. Döntés-előkészítés és transzformációk

A tanulási és keresési hatékonyság növelését szolgáló transzformációs függvények:

- `decision_preparation()`: A játéktábla állapotát egy egyszerűsített vagy kanonikus formára transzformálja (például forgatások, tükrözések vagy egyértelmű védekező/nyerő lépések felvétele révén). Visszatérési értéke a transzformált játékállapot (`Self`) és a transzformáció kódolási kulcsa (`int`).
- `decision_decoder(inner_result, tran)`: A transzformált állapoton hozott döntést (belső eredményt) és a transzformációs kulcsot felhasználva visszakódolja a lépést az eredeti játéktábla koordináta-rendszerébe.

## 3.3. Absztrakt játékosmodell

A szoftverkeretrendszer másik pillérét a döntéshozó egységek — azaz a játékosok — absztrakciója alkotja. Míg az `abstract_game` a környezetállapotot és a szabályrendszert foglalja magába, addig az `abstract_player` osztály a döntési logikát és a tapasztalatok (tanulási adatok) használatát választja el a konkrét játékmenettől.

Ez az elválasztás lehetővé teszi, hogy a különböző típusú játékosok (emberi felhasználó, szabályalapú heurisztika, Monte-Carlo-fakeresés, Q-tanuló ágens) cserélhetők és egymással tetszőleges kombinációban versenyeztethetők legyenek anélkül, hogy a játéklevezető logikát módosítani kellene.

### 3.3.1. A játékosmodell felépítése és feladatai

Az `abstract_player` osztály mint absztrakt alaposztály két fő kötelezettséget ír elő az abból származtatott konkrét játékososztályok számára:

- **Univerzális lépésmeghatározás**: Függetlenül a játékos belső működésétől (legyen az egy összetett faalapú keresés vagy egy egyszerű billentyűzetbeolvasás), az ágensnek a játékkörnyezet aktuális állapota alapján egy szabályos lépéskoordinátával kell visszatérnie.
- **Tapasztalat- és adatinjektálás**: A gépi tanulási modellek használata során szükséges a szimulációk során gyűjtött adatok átadása az ágenseknek. Azok a játékosimplementációk, amik nem használnak adatokat, ezt a függvényt egy üres implementációval (*pass*) valósítják meg.

## 3.4. Függvények specifikációja

- `get_move(game)`: A játékos döntéshozó interfésze. Paraméterül kap egy `abstract_game` állapot-referenciát, amelyből az implementáló játékosok lekérdezhetik a szabályos lépéseket (`get_possible_moves()`), majd egy ezek közül választott lépéskoordináta lesz a visszatérési értékük. Ennek a minimális interfésznek köszönhetően a játékkörnyezet számára a játékos belső architektúrája teljesen rejtve marad, ő csak egy következő lépést kér a játékosoktól.
- `init_data()`: A tapasztalati adatok betöltésére szolgáló interfész. A tanítási szimulációkat koordináló *trainer* komponensek ezen a metóduson keresztül adják át az összegyűjtött tapasztalati adatokat (`experience_data`), amelyeket az RL ágensek felhasználnak a belső modelljük (például a Q-tábla vagy MCTS nyerési aránya) inicializálásához.

# 4. fejezet

# Implementált játékkörnyezetek

## 4.1. Végtelenített amőba

A 3x3-as eltűnő jeles amőba (nemzetközi nevén gyakran *infinite tic-tac-toe*) a klasszikus amőba (*tic-tac-toe*) egy izgalmas, pörgős változata. A játék lényege, hogy a táblán sosem alakulhat ki döntetlen, mert a jelek egy idő után eltűnnek.

### Játékszabályok

#### A játék célja

Ugyanaz, mint a klasszikus amőbában: alakíts ki egy 3 elemű egyenest a saját jeledből vízszintesen, függőlegesen vagy átlósan.

#### Alapszabályok

- **Kezdés**: A játék egy üres, 3x3-as rácson indul. A játékosok felváltva teszik le a jeleiket (x és ∘).
- **Eltűnő jelek**: Egy játékosnak legfeljebb 3 jele lehet egyszerre a táblán, ezért amikor lekerül a negyedik jele, a legrégebben lerakott jele azonnal eltűnik.

## 4.2. Megoldandó problémák

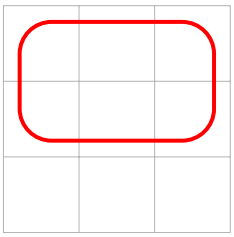
A legfontosabb megoldandó probléma, hogy egy bizonyos állás 8-féleképpen állhat elő (az összes lehetséges transzformációt figyelembe véve). Vagyis ha a gyakorlatilag egyforma eseteket nem különítjük el, akkor a programnak 8-szor annyit kell tanulnia. A megoldás kulcsa, hogy minden egyes táblaálláshoz egy egyértelműen előállítandó tájolást határozzunk meg.

Ennek módszere a következő:

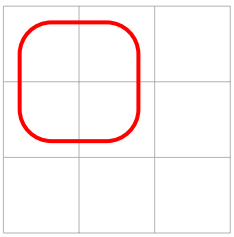
1. A jeleket számokkal helyettesítjük, hogy számon lehessen tartani, hogy melyik a legöregebb. A számok kettő hatványok lesznek, mert összegeket képzünk belőlük, és a cél, hogy egy bizonyos összeg pontosan egyféleképpen állhasson elő. Továbbá 2-től kezdjük a számozást, mert az 1-est fenntartjuk érvényes lépés jelölésére egy üres mezőben. Így:

   - ○: 2, 4, 8
   - x: 16, 32, 64

2. A 3x3-as tábla mind a négy 2x3-as blokkjában meghatározzuk a jelek számértékeinek összegét. Ezt követően a táblát úgy fogjuk forgatni, hogy a legnagyobb összeggel rendelkező blokk a felső 2x3-as helyre kerüljön.
   Ábra:

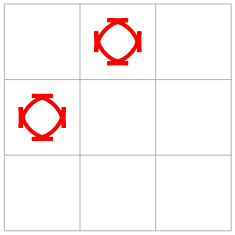


3. Az így felülre forgatott 2x3-as blokknak összegezzük a bal- és jobb oldali 2x2-es blokkjait, majd úgy tükrözzük az adott állást, hogy a nagyobb összegű blokk a bal felső 2x2-es legyen.
   Ábra:



Mivel a jeleket 2 hatványokkal kódoltuk, ezért egy összeg felfogható egy 2-es számrendszerben felírt számnak, vagyis egy bizonyos összeg pontosan egyféleképpen állítható elő. Ezért két blokk összege csak akkor lehet egyenlő, ha ugyanazok a számok szerepelnek bennük. Ami viszont a maximális 6 jel esetében nem lehetséges, mert két 2x3-as blokknak legfeljebb 3 közös eleme lehet. Továbbá a harmadik lépéstől a 2x2-es blokkok sem lehetnek egyforma összegűek, mert egy 2x3-as blokkba legalább 3 jelnek bele kell esnie, és két 2x2-es blokknak csak 2 közös eleme van, vagyis biztosan lesz valamelyik blokkban még legalább egy szám, ami nem a kettejük metszetébe esik. Vagyis ez a módszer a 6 jeles állásokhoz egy egyértelmű tájolást rendel hozzá.

További megfontolást igényelnek azok az esetek, amikor még nincs letéve mindkét játékos mindhárom jele. Ezekben az esetekben lehetségesek olyan esetek, amikor több egyforma összegű 2x3-as blokkok vannak. Ekkor meghatározzuk mind a négy 2x2-es blokk összegét, amelyek közül kiválasztjuk az első legnagyobbat (mert még itt is lehet több egyforma), és a táblát a 3. pontnak megfelelően úgy forgatjuk, hogy a bal felső sarokba essen. Ezután megvizsgáljuk az ábrán jelölt két mező értékét és szükség szerint tükrözzük, hogy a felső középső elem legyen a nagyobb.
Ábra:



Így szintén egy egyértelmű tájolást sikerült elérni.

### 4.2.1. Transzformálás

A táblaállást egy 9 elemű sorvektorként kezeljük (szükség esetén másolást követően 3x3-as mátrixszá alakítjuk és úgy dolgozunk vele tovább), így egy transzformáció megfeleltethető egy 9x9-es mátrixszal való szorzásnak. Az egységes tájolásra traszformálást jobbról való szorzásnak vesszük, így a visszatranszformálás egy ugyanazzal a mátrixszal való balról (oszlopvektort) való szorzás lesz.

Ezek a transzformációk, a tájolás megkeresése, valamint a táblatranszformálás és a visszatranszformálás is a `infinite_tictactoe_transformations_matrices.py` és `infinite_tictactoe_transformations.py` file-okban találhatók meg.

### 4.2.2. Szimmetriák kezelése

Mivel az egyes jelek nem egyforma értékűek, így a maximális 6 jel esetében nem alakulhat ki szimmetrikus állás. Viszont az első három jel rakása esetén még igen. Mivel az egységes tájolást úgy végezzük, hogy a felső 2x3-as illetve a bal felső 2x2-es blokkhoz viszonyítunk mindent, így a szimmetriákat arra a két esetre kell megvizsgálnunk (az egységes tájolásra transzformálás után), hogy a függőleges szimmetriatengelyre, illetve a főátlóra nézve szimmetrikus-e az adott állás. Amennyiben:

- **Függőleges tengelyre szimmetrikus**: A jobb oldali oszlopban levő mezőket kivesszük a választandó lehetőségek közül.
- **Főátlóra szimmetrikus**: Csak a felső háromszögben levő mezők lesznek választható mezők.

Természetesen csak a szabad mezőket korlátozzuk. Ezt az `mask_symmetric` függvény végzi. Ezzel az utolsó lehetőséget is kiszűrtük, hogy egyforma lépéseket külön esetként vizsgáljon.

### 4.2.3. Lehetséges lépések terének csökkentése

Minden esetben csak 3 lehetséges lépése van, azonban vannak esetek, amikor egyértelmű a nyerő lépés. Mivel minden lépés során a legrégebbi jel eltűnik, ezért a nyerő lépés vizsgálatánál a táblát kezdettől fogva úgy vizsgáljuk, hogy a legrégebbi jelet (amelyik játékos szemszögéből vizsgáljuk) kitöröljük, majd megvizsgáljuk, hogy valamelyik sorban, oszlopban vagy átlóban van-e olyan szabad hely, amellyel 3 egyforma jelet lehet kialakítani. Ezt a `get_force_win()` függvény végzi. Ennél több taktikai lépés beépítése nem történt és nem is szükséges.

## 4.3. Állás elmentése

Az egyes állásokat az egységes tájolásra transzformált állapotban mentjük el, úgy, hogy a táblaállásnak megfelelő számokat folytonosan egy *string*be írjuk (az üres helyeket 0-val jelölve), és a soron következő játékos jelének megfelelő számot az utolsó helyen hozzátoldjuk. Ugyanis (mind a 3-3 jel lent léte esetén) mindegyik állás megvalósítható úgy, hogy vagy a ∘ vagy az x játékos következzen.

A tanulóalgoritmus ágensek a későbbiekben ezeket a táblaállásoknak megfelelő *string*eket fogják szótárakban kulcsként használni.

# 5. fejezet

# Implementált ágensek és játékosmodellek

## 5.1. Minimax algoritmus

## 5.2. Monte-Carlo-fakeresésen (MCTS) alapuló ágens

A Monte-Carlo-fakeresés (Monte Carlo Tree Search, MCTS) elméleti hátterét és működési lépéseit a szakirodalmi áttekintésben részletesen tárgyaltuk. Jelen alfejezet az elmélet gyakorlati leképezésére fókuszál: bemutatja, hogyan épül fel az MCTS-alapú ágens az `abstract_player` interfész mentén, és milyen specifikus logikát alkalmaz kétszemélyes absztrakt logikai játékok döntési terének bejárására. Az MCTS-ágens a döntéshozatala során nem egy előre megírt szabályrendszert követ, hanem a rendelkezésre álló erőforráskeretig (meghatározott számú iterációig) ciklikusan ismétli a fakeresés négy alapvető lépését (kijelölés, kiterjesztés, szimuláció, visszaterjesztés).

A fán belüli navigáció során az ágens a kijelölési fázisban az UCT (Upper Confidence Bound Applied to Trees) formulát használja kiértékelő függvényként a gyermekcsomópontok közötti választáshoz:

$$UCT = \frac{w_i}{n_i} + c \cdot \sqrt{\frac{ln(N_i)}{n_i}}$$

Az ágens minden egyes iterációban az UCT-értékekkel súlyozott arányban halad tovább a gyermekcsomópontok között, amíg ki nem kutatott állapothoz nem ér.

### 5.2.1. Megvalósított algoritmus

Az `abstract_player` osztályból származtatott MCTS-ágens lépésválasztásának a folyamata (`get_move()`) függvényen keresztül az alábbi lépésekből áll:

1. **Kényszerített nyerő lépés ellenőrzése**: Az ágens elsőként ellenőrzi, hogy az aktuális állapotban létezik-e azonnali nyerő lépés a `get_force_win()` függvény segítségével. Amennyiben található ilyen, az ágens ezt a lépést választja.

2. **Állapottér-redukció és transzformáció**: Nyerő lépés hiányában a játéktábla állapota a `decision_preparation()` függvénnyel egy (az adott játékhoz jellemzett) kanonikus (egységesített) formára transzformálódik – amennyiben a konkrét játékhoz létezik –, és a szimmetriáknak megfelelően a lehetséges lépéseket leszűkíti.

3. **Transzformált lépések lekérdezése**: Az így kapott, redukált játékállapoton az ágens lekéri az összes lehetséges lépést a `get_possible_moves()` függvény hívásával.

4. **Lépések visszatranszformálása**: Mivel a kapott lépések a belső, transzformált állapotra vonatkoznak, az ágens egy ciklus segítségével a játékhoz tartozó `decision_decoder()` függvény használatával előállítja az eredeti játékállapotnak megfelelő lehetséges lépéseket.

5. **UCT-értékek meghatározása**: Az eredeti játékállapot lehetséges lépéseihez az ágens kiszámítja az UCT-formulának megfelelő értékeket (`mcts_eval()`). A korábban feltárt csomópontokat egy szótárban (*dictionary*) tárolja, ahol a kulcsok a tábla kódolásával (`board2str()`) előállított *string*ek, az értékek pedig kételemű listák: az első elem az addigi összes látogatottságot ($n$), a második a nyert szimulációk számát ($w$) jelöli.

6. **Kiválasztás**: Végül az ágens a kiszámított UCT-értékeknek megfelelő súlyozott valószínűségi eloszlás alapján, véletlenszerűen választ a lehetséges lépések közül.

# A. függelék

# A mellékelt ZIP állomány könyvtárszerkezete

A dolgozathoz tartozó elektronikus melléklet belső könyvtárstruktúrája az alábbi.

```
.
 codes
    games
       abstract_game_display.py
       abstract_game.py
       game_master.py
       infinite_tictactoe
           infinite_tictactoe_basic.py
           infinite_tictactoe_display.py
           infinite_tictactoe_game.py
           infinite_tictactoe_io.py
           infinite_tictactoe_transformations.py
           tictactoe_transformations.py
    main.py
    players
       abstract_player.py
       homo_sapiens_player.py
       mcts_player.py
       mcts_simple_player.py
       minimax_alphabeta_player.py
       minimax_player.py
       random_player.py
    test.py
    trainers
       abstract_trainer.py
       mcts_trainer.py
```

```
    utils
        runtime.py
        side.py
 data
    infinite_tictactoe_mcts_data
 documentation

9 directories, 23 files
```